

Resoluções Didáticas e Científicas —

Capítulo 3

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Abordagem: Progressão Incremental e Rigor de Engenharia de Software

Apresentação Pedagógica

Olá! É uma grande satisfação dar as boas-vindas a você nas resoluções estruturadas do Capítulo 3. Neste capítulo, estabelecemos os blocos de construção fundamentais do desenvolvimento de software profissional através das **estruturas de controle de fluxo** (sequência, seleção e repetição), da metodologia de **refinamento sucessivo top-down** e da formulação algorítmica em pseudocódigo [cite: 543, 629, 832].

Fiel ao nosso compromisso com a **abordagem pedagógica incremental**, todas as resoluções técnicas a seguir restringem-se estritamente aos conceitos apresentados até este ponto (Capítulos 1, 2 e 3). Portanto, utilizaremos apenas a estrutura de repetição while, operadores de atribuição compostos e os operadores de incremento e decremento [cite: 576, 580, 906]. Não faremos uso de estruturas como for, do...while, switch, ponteiros ou matrizes, que serão abordadas no momento oportuno em capítulos futuros.

1. Exercícios de Autorrevisão

Exercício 3.1 a 3.3 — Fundamentos de Fluxo de Controle

- **3.1 Lacunas:** a) Algoritmo [cite: 629]. b) Controle do programa [cite: 629]. c) Sequência, seleção, repetição [cite: 629]. d) if...else [cite: 629]. e) Instrução composta [cite: 629]. f) while [cite: 629]. g) Controlada por contador [cite: 630]. h) Sentinela [cite: 630].
- **3.2 Incremento de Variáveis:** As quatro expressões conceitualmente equivalentes para adicionar 1 a x são: $x = x + 1$; $x += 1$; $++x$; e $x++$; [cite: 631, 632, 633, 634].
- **3.3 Instruções Isoladas:** a) $z = x + y++$; [cite: 636]. b) $\text{produto} *= 2$; [cite: 637]. c) $\text{produto} = \text{produto} * 2$; [cite: 638, 639]. d) ``if (contador > 10) printf("Contador é maior que 10.\n");`` [cite: 640, 641].

Exercício 3.6 — Efeitos Colaterais de Pós-Incremento

Dadas as condições iniciais $\text{produto} = 5$ e $x = 5$, a instrução $\text{produto} *= x++$; resulta nos seguintes valores finais nas posições de memória: **produto = 25** e **x = 6** [cite: 645].

Cientificamente, o operador de pós-incremento ($++$ posicionado à direita da variável) avalia a expressão utilizando o valor atual armazenado em x (5) antes de aplicar o incremento. Assim, a operação executada na CPU é $\text{produto} = 5 * 5$, e imediatamente após a avaliação da

expressão, o efeito colateral é disparado, atualizando x para 6.

Exercício 3.9 — Depuração de Código (Bugs Comuns)

- **a) Bug:** Falta a chave de fechamento correspondente ao bloco do laço while [cite: 727]. **Correção:** Adicionar } após a linha do incremento da variável c [cite: 727].
- **b) Bug:** Uso ilegal de especificação de precisão (%.4f) dentro da string de formato da função de entrada scanf [cite: 728]. **Correção:** Alterar para scanf("%f", &valor); [cite: 729].
- **c) Bug:** Sintaxe incorreta na palavra-chave de controle (uso de "se não" em vez de else) e ponto e vírgula terminal espúrio no bloco do else, gerando um erro de lógica em que o segundo `printf` roda incondicionalmente [cite: 730, 731]. **Correção:** Substituir por else printf("Homem\n"); [cite: 732].

2. Exercícios Regulares (Soluções de Engenharia de Software)

Exercício 3.11 — Correção de Sintaxe de Laço e Seleção

a) Trecho com erro: if (idade >= 65); printf(...);

Análise: O ponto e vírgula terminal encerra a instrução condicional prematuramente, isolando o bloco do else subsequente e disparando um erro de compilação [cite: 737, 739].

Correção: if (idade >= 65) printf("Idade é maior ou igual a 65\n"); else printf("Idade é menor que 65\n"); [cite: 738, 740]

b) Trecho com erro: while (x <= 10) { } total += x; ++x;

Análise: As chaves vazias criam um bloco de repetição sem corpo útil. Como x inicia em 1, a condição x <= 10 será infinitamente verdadeira, gerando travamento do sistema [cite: 741, 742].

Correção: while (x <= 10) { total += x; ++x; } [cite: 744, 745]

Exercício 3.13 — Rastreamento de Memória (Análise de Saída)

O programa executa um laço controlado por contador de 1 a 10, computando o quadrado de cada iteração e acumulando na variável total [cite: 776, 777, 778]. A saída impressa linha por linha na tela é:

```
1
4
9
16
25
36
49
64
81
100
Total is 385
```

Cientificamente, o valor final é derivado da soma dos quadrados dos dez primeiros inteiros positivos: $\sum_{x=1}^{10} x^2 = 385$.

Exercício 3.17 — Programa de Monitoramento de Consumo de Combustível

Este sistema de software aplica o conceito de **acumulação controlada por sentinela** [cite: 571], permitindo processar um número indefinido de abastecimentos até que o usuário digite o marcador de saída -1 [cite: 837, 850].

```
#include <stdio.h>
```

```
int main(void) {
    float litros, km;
    float total_litros = 0.0;
    float total_km = 0.0;

    printf("Informe quantos litros abasteceu (-1 para terminar): ");
    scanf("%f", &litros);

    while (litros != -1.0) {
        printf("Informe quantos km dirigiu: ");
        scanf("%f", &km);

        printf("O consumo atual é de %f km/\n", km / litros);

        total_litros += litros;
        total_km += km;

        printf("Informe quantos litros abasteceu (-1 para terminar): ");
        scanf("%f", &litros);
    }

    if (total_litros != 0.0) {
        printf("O consumo geral foi de %f km/\n", total_km / total_litros);
    }

    return 0;
}
```

Dica de Prevenção de Erros 3.1: Sempre teste se o divisor (neste caso, `total\_litros`) é diferente de zero antes de realizar uma operação de divisão, prevenindo a quebra catastrófica do programa por exceção de ponto flutuante (divisão por zero) em tempo de execução.

Exercício 3.18 — Sistema de Gestão de Limite de Crédito Comercial

```
#include <stdio.h>
```

```

int main(void) {
    int conta;
    float saldo_inicial, encargos, creditos, limite_credito, novo_saldo;

    printf("Informe o número da conta (-1 para terminar): ");
    scanf("%d", &conta);

    while (conta != -1) {
        printf("Informe o saldo inicial: ");
        scanf("%f", &saldo_inicial);
        printf("Informe o total de encargos: ");
        scanf("%f", &encargos);
        printf("Informe o total de créditos: ");
        scanf("%f", &creditos);
        printf("Informe o limite de crédito: ");
        scanf("%f", &limite_credito);

        novo_saldo = saldo_inicial + encargos - creditos;

        if (novo_saldo > limite_credito) {
            printf("Conta: %d\n", conta);
            printf("Limite de crédito: %.2f\n", limite_credito);
            printf("Saldo: %.2f\n", novo_saldo);
            printf("Limite de crédito ultrapassado.\n");
        }

        printf("\nInforme o número da conta (-1 para terminar): ");
        scanf("%d", &conta);
    }

    return 0;
}

```

Exercício 3.21 — Calculadora Estatística de Salários Semanais

Este algoritmo mapeia as regras corporativas de pagamento com acréscimo de 50% (hora e meia) para todas as horas laboradas que excederem o teto fixo regulamentar de 40 horas semanais [cite: 900].

```
#include <stdio.h>
```

```

int main(void) {
    int horas;
    float taxa_hora, salario;

    printf("Digite # de horas trabalhadas (-1 para terminar): ");
    scanf("%d", &horas);

```

```

while (horas != -1) {
    printf("Digite o salário por hora do funcionário (R$00,00): ");
    scanf("%f", &taxa_hora);

    if (horas <= 40) {
        salario = horas * taxa_hora;
    }
    if (horas > 40) {
        salario = (40 * taxa_hora) + ((horas - 40) * taxa_hora * 1.5);
    }

    printf("Salário é de R$%.2f\n\n", salario);

    printf("Digite # de horas trabalhadas (-1 para terminar): ");
    scanf("%d", &horas);
}

return 0;
}

```

Exercício 3.24 — Algoritmo de Busca de Máximo (Determinação de Maior Inteiro)

Pseudocódigo Formal:

Inicializar contador em 1

Ler o primeiro número e atribuir à variável maior

Enquanto contador for menor ou igual a 9:

Ler o próximo número

Se o número lido for maior que o valor armazenado em maior:

maior = número

Incrementar contador

Imprimir o valor de maior

#include <stdio.h>

```

int main(void) {
    int contador = 1;
    int numero, maior;

    printf("Digite o número 1: ");
    scanf("%d", &numero);
    maior = numero; /* Primeiro elemento é assumido como maior inicial */

    while (contador < 10) {
        printf("Digite o número %d: ", contador + 1);
        scanf("%d", &numero);
    }
}

```

```

        if (numero > maior) {
            maior = numero;
        }

        contador++;
    }

    printf("O maior número achado é: %d\n", maior);
    return 0;
}

```

Exercício 3.35 — Testador Científico de Números Palíndromos

Análise Algorítmica: Para verificar se um inteiro de 5 dígitos é um palíndromo sem o auxílio de vetores, devemos isolar matematicamente as extremidades numéricas através de sucessivos isolamentos aritméticos usando divisões e restos inteiros de base 10 [cite: 1123, 1124].

```
#include <stdio.h>
```

```

int main(void) {
    int numero, original;
    int d1, d2, d4, d5;

    printf("Digite um número de cinco dígitos: ");
    scanf("%d", &numero);
    original = numero;

    /* Isolamento matemático dos dígitos individuais */
    d1 = numero / 10000;      /* Primeiro dígito (extrema esquerda) */
    d2 = (numero / 1000) % 10; /* Segundo dígito */
    d4 = (numero / 10) % 10;   /* Quarto dígito */
    d5 = numero % 10;         /* Quinto dígito (extrema direita) */

    /* Um número de 5 dígitos é palíndromo se d1 == d5 E d2 == d4 */
    if (d1 == d5) {
        if (d2 == d4) {
            printf("O número %d é um PALÍNDROMO.\n", original);
        }
    }

    if (d1 != d5) {
        printf("O número %d NÃO é um palíndromo.\n", original);
    }

    if (d1 == d5) {
        if (d2 != d4) {
            printf("O número %d NÃO é um palíndromo.\n", original);
        }
    }
}

```

```

    }
}

return 0;
}

```

3. Seção Fazendo a Diferença

Exercício 3.49 — Criptografia e Proteção de Dados (Padrão Industrial de Transmissão)

Lógica Matemática da Codificação: O algoritmo de criptografia processa inteiros escalares de quatro dígitos recebidos separadamente através de transformações lineares lineares invariantes [cite: 1173, 1174]: $D_{\text{cripto}} = (D_{\text{original}} + 7) \bmod{10}$. Posteriormente, efetua um rearranjo estrutural de chaves (posições 1 com 3, e 2 com 4) [cite: 1174].

Programa de Criptografia:

```

#include <stdio.h>

int main(void) {
    int numero, d1, d2, d3, d4, temp;

    printf("Digite o inteiro de quatro dígitos a ser criptografado: ");
    scanf("%d", &numero);

    /* Decomposição */
    d1 = numero / 1000;
    d2 = (numero / 100) % 10;
    d3 = (numero / 10) % 10;
    d4 = numero % 10;

    /* Transformação matemática */
    d1 = (d1 + 7) % 10;
    d2 = (d2 + 7) % 10;
    d3 = (d3 + 7) % 10;
    d4 = (d4 + 7) % 10;

    /* Permutação de dígitos (1-3, 2-4) */
    temp = d1; d1 = d3; d3 = temp;
    temp = d2; d2 = d4; d4 = temp;

    printf("Código Criptografado: %d%d%d%d\n", d1, d2, d3, d4);
    return 0;
}

```

Programa de Decriptografia (Inversão Lógica da Criptografia):

Para inverter o processo de criptografia baseado na operação modular $+7 \pmod{10}$, a CPU executa a operação aritmética espelho. Como a subtração simples pode produzir números negativos em C (o que afeta a operação do resto), adicionamos um ciclo completo de base decimal (+10), gerando a equação estável de decodificação: $D_{\text{original}} = (D_{\text{cripto}} + 10 - 7) \pmod{10} = (D_{\text{cripto}} + 3) \pmod{10}$.

```
#include <stdio.h>
```

```
int main(void) {
    int codigo, d1, d2, d3, d4, temp;

    printf("Digite o código de quatro dígitos a ser decodificado: ");
    scanf("%d", &codigo);

    /* Decomposição do código criptografado */
    d1 = codigo / 1000;
    d2 = (codigo / 100) % 10;
    d3 = (codigo / 10) % 10;
    d4 = codigo % 10;

    /* Desfazer a permutação original (reverter posições 1-3 e 2-4) */
    temp = d1; d1 = d3; d3 = temp;
    temp = d2; d2 = d4; d4 = temp;

    /* Desfazer a transformação modular matemática */
    d1 = (d1 + 3) % 10;
    d2 = (d2 + 3) % 10;
    d3 = (d3 + 3) % 10;
    d4 = (d4 + 3) % 10;

    printf("Número Original Recuperado: %d%d%d%d\n", d1, d2, d3, d4);
    return 0;
}
```

4. Código-Fonte LaTeX Master Completo

```
\documentclass[12pt]{article}
\usepackage[utf8]{inputenc}
\usepackage[brazil]{babel}
\usepackage{amsmath}
\usepackage{booktabs}
\usepackage{listings}
\usepackage{geometry}
\geometry{a4paper, margin=1in}
```



```
\lstset{
  language=C,
  basicstyle=\ttfamily\small,
  breaklines=true,
  keywordstyle=\bfseries,
  commentstyle=\itshape,
  showstringspaces=false
}
```

```
\title{Resoluções Cientes e Didáticas -- Capítulo 3\ \large C: Como Programar
(6\textordfeminine\ Ed.\ Deitel)}
\author{Prof. Paul Deitel}
\date{\today}
```

```
\begin{document}
\maketitle
```

```
\section{Exercícios de Autorrevisão}
```

```
\subsection*{Exercício 3.1 -- Lacunas}
```

a) Algoritmo. b) Controle do programa. c) Sequência, seleção, repetição. d) if...else.
e) Instrução composta. f) while. g) Controlada por contador. h) Sentinela.

```
\subsection*{Exercício 3.6 -- Rastreamento de Pós-Incremento}
```

A instrução `produto *= x++;` com valores iniciais iguais a 5 resulta em:

```
\begin{equation*}
\text{produto} = 25, \quad x = 6
\end{equation*}
```

```
\section{Exercícios Regulares e Códigos Funcionais}
```

```
\subsection*{Exercício 3.17 -- Consumo de Gasolina}
```

```
\begin{lstlisting}
```

```
#include
```

```
int main(void) {
```

```
    float litros, km, t_litros = 0, t_km = 0;
```

```
    printf("Informe quantos litros abasteceu (-1 para terminar): ");
```

```
    scanf("%f", &litros);
```

```
    while (litros != -1.0) {
```

```
        printf("Informe quantos km dirigiu: ");
```

```
        scanf("%f", &km);
```

```
        printf("O consumo atual e de %f km/\n", km / litros);
```

```
        t_litros += litros; t_km += km;
```

```
        printf("Informe quantos litros abasteceu (-1 para terminar): ");
```

```
        scanf("%f", &litros);
```

```
    }
```

```
    if (t_litros != 0.0) {
```

```
        printf("O consumo geral foi de %f km/\n", t_km / t_litros);
```

```
    }
```

```

    return 0;
}
\end{lstlisting}

```

\subsection*{Exercício 3.35 -- Testador de Palíndromos}

```

\begin{lstlisting}
#include
int main(void) {
    int numero, d1, d2, d4, d5;
    printf("Digite um numero de cinco digitos: ");
    scanf("%d", &numero);
    d1 = numero / 10000;
    d2 = (numero / 1000) % 10;
    d4 = (numero / 10) % 10;
    d5 = numero % 10;
    if (d1 == d5 && d2 == d4) {
        printf("E um PALINDROMO.\n");
    } else {
        printf("NAO e um palindromo.\n");
    }
    return 0;
}
\end{lstlisting}

```

\section{Seção 3.4 -- Fazendo a Diferença}

\subsection*{Exercício 3.49 -- Criptografia}

A criptografia baseia-se na transformação $D_{\text{cripto}} = (D_{\text{original}} + 7) \pmod{10}$, seguida pela inversão das chaves de posição. Para a decodificação, aplica-se a operação complementar estável e livre de resultados lógicos negativos: $D_{\text{original}} = (D_{\text{cripto}} + 3) \pmod{10}$.

\end{document}